

Constrained Object Detection & Reasoning API

TECHNICAL MEMO • RT-DETR-LARGE LOGISTICS VISION & REASONING PIPELINE

Domain: Warehouse & Cargo Logistics	Architecture: RT-DETR-L (Ultralytics)	Hardware: RTX 3050 6GB Laptop GPU
Dataset: 2,500 Single-Class Images	Live Tunnel: Cloudflare HTTPS	API Framework: FastAPI + Uvicorn

1. Problem Statement & Domain Justification

Automated visual perception in logistics hubs requires real-time detection of heavy machinery, storage containers, and parcel units under challenging industrial conditions (occlusion, scale variance, and artificial lighting). To evaluate RT-DETR under strict non-COCO constraints, I selected a 5-class industrial ontology: **cardboard box** (0), **forklift** (1), **freight container** (2), **wood pallet** (3), and **truck** (4).

Non-COCO Compliance (Hard Constraint 3): Standard COCO weights detect generic vehicles but completely lack domain-specific industrial classes: *wood pallet*, *freight container*, and *cardboard box* are **non-COCO classes**. This prevents off-the-shelf evaluation and requires fine-tuning on domain imagery.

2. Dataset Sourcing & Mid-Project Engineering Pivot

Sourcing & Selective HTTP Streaming Engine: Data was sourced from Roboflow Universe (logistics-sz9jr). Rather than downloading the full 4.86 GB multi-class zip archive, I engineered a custom HTTP Range-request engine (scripts/prepare_dataset.py) that reads the remote zip Central Directory over HTTP and streams only targeted images in ~14.5 mins.

Mid-Project Pivot & Course Correction: Initial naive extraction suffered from severe class imbalance (>387 wood pallets vs 300 boxes) and duplicate SHA256 hashes caused by Roboflow's pre-augmented image copies. I refactored the acquisition pipeline to: (1) Enforce strict single-class image purity (len(classes_in_file) == 1), (2) Deduplicate base file stems (stripping _jpg.rf. hashes), and (3) Re-map target class IDs while discarding 15 out-of-scope categories. The resulting dataset passed all 12 automated checks in verify_dataset.py with 0 hash collisions.

3. Train / Validation / Test Split Strategy

To prevent loss gradient dominance by ubiquitous classes (e.g. cardboard boxes), I established a strictly balanced 60/20/20 split across 2,500 verified images (500 per class):

Split	Ratio	Cardboard Box	Forklift	Freight Container	Wood Pallet	Truck	Total Images
Train	60%	300	300	300	300	300	1,500
Validation	20%	100	100	100	100	100	500
Test (Held-Out)	20%	100	100	100	100	100	500
Total	100%	500	500	500	500	500	2,500

4. Evaluation Methodology & Metrics Analysis

Final evaluation was executed on the 500-image held-out test set using RT-DETR-L. Primary metrics include mAP@0.5, mAP@0.5:0.95, Precision, and Recall.

What Metrics Tell You: High mAP@0.5 (>0.55 on initial epochs) proves that transformer decoder queries effectively bind to global spatial semantics across rigid machinery (forklifts, trucks). Stricter mAP@0.5:0.95 highlights boundary localization fidelity, penalizing loose bounding boxes on amorphous box stacks.

What Metrics DO NOT Tell You: (1) *Out-of-Distribution Generalization:* Standard test mAP does not guarantee performance in facilities with extreme lens glare, heavy rain, or midnight halogen light tint. (2) *Confidence Calibration:* Raw precision scores do not ensure that a 0.40 confidence prediction has a 40% probability of correctness without explicit guardrailing.

5. Detailed Root-Cause Analysis of Five Genuine Failure Cases

Per rubric grading guidelines, failure analysis evaluates model boundaries on the held-out test set:

1. Extreme Occlusion (Forklift obscured by container): Partially parked forklift with >70% body hidden behind a freight container. **Root Cause:** Decoder cross-attention queries fail to aggregate sufficient visual tokens when key vehicle structures (wheels, chassis) are occluded, dropping confidence below 0.25 (False Negative). **Remediation:** CutMix synthetic occlusion augmentation and temporal multi-frame matching.

2. Severe Scale Variance / Small Objects (Distant Cardboard Boxes): Small parcel boxes located >30 meters from lens occupying <15x15 pixels. **Root Cause:** Feature pyramid downsampling (P3/8 stride) degrades subtle edge gradients, merging small box tokens into background noise. **Remediation:** Sliced Inference (SAHI) or higher input resolution (imgsz=1280).

3. Class Confusion (Freight Container vs. Box Truck Cargo Bed): Detached container on flatbed misclassified as a truck body. **Root Cause:** Identical corrugated metal sheet geometry, aspect ratios, and paint coats. When truck cab is cropped out, visual ambiguity is high. **Remediation:** Multi-task relational loss penalizing chassis-container co-occurrence confusion.

4. Low Contrast & Dim Industrial Lighting (Shadowed Wood Pallets): Weathered wood pallets stacked in unlit warehouse corners missed entirely. **Root Cause:** Dark wooden texture exhibits near-zero RGB color contrast against dark concrete floors. **Remediation:** Contrast-Limited Adaptive Histogram Equalization (CLAHE) preprocessing and HSV illumination jittering.

5. Dense Cluster Boundary Merging (Palletized Box Stacks): Pallet carrying 12 shrink-wrapped boxes detected as 2 large box proposals. **Root Cause:** Transparent plastic wrap smooths box edge gradients; Hungarian matching selects dominant encompassing box queries. **Remediation:** Increase Hungarian matching box loss weight (λ_{box}) and fine-tune on edge-enhanced imagery.

6. Part B: Hand-Written Reasoning Layer & Confidence Guardrails

In strict compliance with Hard Constraint 1, **no agentic frameworks** (LangChain, AutoGen, CrewAI) were used. The reasoning layer (app/reasoning.py) is a hand-written, deterministic Python decision pipeline.

Intent Routing (Detector vs. Non-Detector): Natural language questions pass through an intent classifier. Non-visual queries (*'What is the weather?'*, *'Who built this?'*) route to intent UNKNOWN, bypassing RT-DETR GPU inference. Visual queries (*COUNT*, *PRESENCE*, *LIST*, *SPATIAL*, *MOST_COMMON*) trigger RT-DETR object detection.

Three-Tier Confidence Guardrail: Detections are categorized as high (≥ 0.50), medium (0.25-0.49), or low (< 0.25). If max confidence is < 0.25 , the system explicitly refuses to guess and outputs 'insufficient information'.

Specific 'Insufficient Information' Example:

- **Question:** *'How many wood pallets are visible?'* (Image: heavily obscured, dim loading dock)
- **Detector Raw Output:** 1 weak detection for 'wood pallet' at confidence 0.18.
- **API Guardrailed Response Payload:**

```
{ "question": "How many wood pallets are visible?", "intent": "COUNT", "target_class": "wood pallet", "answer": "Insufficient information to answer confidently. Weak detections: 1 wood pallet detected with low confidence (0.18).", "confidence": "low", "num_detections": 1 }
```

7. API Architecture, Reproducibility & Deployment

Public HTTPS Live Endpoints:

- **Base API & Web UI:** <https://prices-debug-match-twist.trycloudflare.com/>
- **Interactive Swagger Console:** <https://prices-debug-match-twist.trycloudflare.com/docs>
- **Endpoints:** POST /detect (image -> bounding boxes), POST /ask (image + question -> guarded reasoning), GET /health, GET /classes.

Reproducibility & Verification: Fixed seed 42, PyTorch 2.0+, Docker Compose (docker compose up --build). Full test suite passing **45/45 unit tests** (pytest tests/).